

Blink "Engenharia de Requisitos para Devs"



Blink "Engenharia de Requisitos para Devs"

Alberto Souza

2026-02-23

Contents

O que você vai aprender com este curso?	i
A qualidade do seu código começa muito antes da primeira linha	1
Requisitos incompletos são o problema mais citado no mundo inteiro	3
Triturar requisitos é a forma mais barata de reduzir custo operacional	5
O Shape Up oferece o melhor dos dois mundos entre requisitos formais e ágeis	7
Busque acesso a quem idealizou o requisito — essa é a fonte mais confiável	9
Entenda o motivo de existência do requisito antes de pensar em como implementá-lo	11
Faça double check do seu entendimento — aprendiza- gem negativa é um risco real	13
Rascunhe o fluxo antes de codar para revelar problemas escondidos	15
Especifique cenários de teste antes de codar — é mais uma camada de reflexão	17
Se você não consegue imaginar o que vai codar, como vai executar?	19
Resumo Final	21

Sobre este resumo 23

O que você vai aprender com este curso?

Você já parou para pensar em quantas vezes implementou algo que depois precisou ser refeito porque o requisito não estava claro? Quantas horas de retrabalho poderiam ter sido evitadas se, antes de abrir a IDE, você tivesse investido mais tempo entendendo e validando o que de fato precisava ser feito?

Neste curso, Alberto Souza apresenta uma abordagem prática de engenharia de requisitos voltada especificamente para pessoas desenvolvedoras. A ideia central é poderosa: **codar é a última etapa de um processo de reflexão**, e quanto mais preparado você chegar para essa etapa, menor será o retrabalho e maior a qualidade da entrega. Inspirado pelo método Shape Up e sustentado por pesquisas como o artigo “Naming the Pain in Requirements Engineering”, o curso entrega um método de cinco passos para triturar requisitos antes de escrever uma única linha de código.

Ao longo destes blinks, você vai descobrir:

- Por que **requisitos mal refinados** são o problema número um em projetos de software — inclusive nos que se dizem ágeis.
- Como a **qualidade do seu código** depende de muito mais do que design e testes — ela começa no requisito.
- Por que o método **Shape Up** oferece uma ponte entre engenharia de requisitos formal e práticas ágeis.
- Como **buscar acesso à fonte original** do requisito é a primeira coisa que você deve fazer.
- Por que **interrogar o motivo de existência** de um requisito pode poupar semanas de trabalho.
- Como o **double check do seu entendimento** funciona como

feedback de aprendizagem.

- Por que **rascunhar o fluxo** antes de codar revela problemas escondidos.
- Como **especificar cenários de teste** antes da implementação fortalece a reflexão.
- Por que **descrever o passo a passo da implementação** é o último filtro antes de codar.
- E como transformar tudo isso em um **template reutilizável** que eleva o padrão da sua equipe.

Se você quer parar de apagar incêndios e começar a preveni-los, este resumo vai te dar as ferramentas práticas para isso.

A qualidade do seu código começa muito antes da primeira linha

A maioria das pessoas desenvolvedoras investe pesado em aprender design de código, frameworks, bibliotecas e testes automatizados. E está certo — tudo isso é fundamental. Mas existe um elemento anterior a tudo isso que influencia diretamente a qualidade do que você entrega: o requisito.

Pense assim: a qualidade do seu código é o acúmulo de diversas coisas bem feitas. Você precisa dominar a stack que utiliza para aproveitar o que já está pronto, evitando reescrever código do zero que será menos confiável. Precisa conhecer bem sua IDE para ser mais produtivo. Precisa saber escrever bons testes automatizados para aumentar a confiabilidade. Mas nenhuma dessas habilidades compensa um requisito mal escrito. Se você implementa perfeitamente algo que não era para ser feito daquele jeito, o resultado é retrabalho — não importa o quão elegante seja o código.

O ponto central é que **codar é a última etapa de um fluxo** que transforma uma ideia em software. Essa ideia pode ser uma nova feature, a correção de um bug ou a alteração de uma funcionalidade existente. A galera gosta de chamar isso de “idea to value”. E para percorrer esse caminho com eficiência, o requisito precisa ser refinado, validado e triturado antes de virar código.

“Refinar bem, triturar bem um requisito é a maneira mais barata de diminuir o custo operacional da implementação como um todo.”

Da próxima vez que você receber uma tarefa, antes de abrir a IDE, pergunte-se: eu entendo profundamente o que precisa ser feito aqui? Se a resposta não for um “sim” convicto, você ainda não

está pronto para codar.

Requisitos incompletos são o problema mais citado no mundo inteiro

Existe uma crença comum de que adotar métodos ágeis resolve os problemas de requisitos. A pesquisa “Naming the Pain in Requirements Engineering” — um estudo global com empresas de diversos países, incluindo Austrália, Alemanha, América do Sul e América do Norte — mostra que essa crença não se sustenta. Os mesmos problemas aparecem independentemente do método utilizado.

O problema mais citado na pesquisa, com 109 menções, é o de **requisitos incompletos ou escondidos** (“Incomplete and/or Hidden Requirements”). Logo atrás vem a **falha de comunicação entre equipe do projeto e cliente** — que, por sua vez, foi apontada como a principal causa de falha em projetos. Se você somar os requisitos incompletos com os “under specified requirements that are too abstract”, o peso desse problema fica ainda mais evidente.

O dado mais revelador é que a maioria esmagadora das empresas que responderam à pesquisa se declara ágil. Ou seja, não é porque você tem sprints, backlog e daily meetings que o problema de requisitos desaparece. Ele continua ali, corroendo a qualidade das entregas por dentro.

Uma das causas apontadas pelo artigo é que as pessoas que trabalham refinando requisitos são menos preparadas do que deveriam. Pessoas desenvolvedoras estudam muito sobre programação, design de código e frameworks, mas quase nunca investem tempo estudando sobre engenharia de requisitos.

“Quantas vezes você proativamente, por interesse próprio, leu um artigo científico sobre engenharia de requisito, comprou um livro sobre isso, assistiu uma palestra? A maioria das pessoas não fez.”

Não espere que o método ágil resolva magicamente o problema de requisitos. Se a sua equipe sofre com retrabalho constante, a causa provavelmente está antes do código — no refinamento insuficiente do que precisa ser feito.

Triturar requisitos é a forma mais barata de reduzir custo operacional

A maioria das pessoas pensa em testes automatizados como a principal ferramenta para aumentar a confiabilidade do software. E testes são de fato essenciais. Mas existe algo que vem antes e que pode ter um impacto ainda maior: o trabalho de refinamento e validação do requisito.

Quando você tritura um requisito — ou seja, refina, questiona, valida e detalha cada aspecto do que precisa ser feito — você consegue vários benefícios simultaneamente. Primeiro, fica mais nítido tudo que precisa ser feito, permitindo que você programe com mais tranquilidade. Segundo, um entendimento mais profundo do requisito pode influenciar positivamente o design do seu código. Terceiro, você entrega o que de fato precisa ser entregue, provavelmente em menos tempo e com menos falhas.

Existe um princípio poderoso por trás disso: **quanto mais próximo da origem do fato você realiza o trabalho, mais fácil é de fazer algo com qualidade**. Se você descobre um problema no requisito antes de codar, o custo de correção é mínimo — é uma conversa, um ajuste no documento. Se você descobre o mesmo problema depois de implementar, testar e colocar em produção, o custo explode: tem que repriorizar, refazer, retestar, replantar. É como o recall de um carro — seria muito melhor ter descoberto o defeito na fábrica do que depois de milhares de unidades vendidas.

“Uma vez que o software tá na rua e um problema aparece, o custo pra resolver aquele problema é maior. Tem que pegar e repriorizar alguma coisa, coloca na frente de outra que também era impor-

| *tante."*

Pense no refinamento de requisitos como um investimento com retorno garantido. Cada hora gasta triturando um requisito antes de codar pode economizar dias de retrabalho depois da implementação.

O Shape Up oferece o melhor dos dois mundos entre requisitos formais e ágeis

Se você já se frustrou com a rigidez da engenharia de requisitos formal e também com a falta de estrutura dos requisitos ágeis, o método Shape Up pode ser a resposta. Criado pelas pessoas que trabalham na 37signals (empresa por trás do Basecamp), o Shape Up é um livro curto, prático e com um storytelling que torna a leitura agradável.

O Shape Up divide o processo em três fases: **Shaping**, **Betting** e **Building**. A fase que mais interessa para pessoas desenvolvedoras é o Shaping — o processo de refinar o que precisa ser feito para minimizar surpresas na implementação. O objetivo é claro: quando você sentar para codar, a sensação deve ser “eu já sei boa parte das coisas que vou fazer; meu ato de codar agora é o ato de materializar o que a gente já combinou antes”.

O que torna o Shape Up especial é que ele combina o rigor da engenharia de requisitos formal — descobrir o que precisa, envolver stakeholders, cruzar informações — com a leveza e praticidade dos métodos ágeis. Você não precisa gerar documentos enormes nem manter artefatos que ficam obsoletos em dias. Em vez disso, você shapeia o requisito até que ele esteja maduro o suficiente para ser apostado (betting) e, depois, construído (building) por um time pequeno.

A ideia não é que você adote o Shape Up inteiro como método. A proposta é que você incorpore a **mentalidade do shaping** no seu dia a dia como pessoa desenvolvedora: antes de codar, shapear o requisito profundamente.

“Quando eu li o Shape Up, fiquei muito impressionado. É um livro que cobre aspectos diferentes do fluxo de desenvolvimento e que traz uma junção muito boa do que é escrito na engenharia de requisitos formais com o olhar mais ágil.”

Você não precisa adotar o Shape Up como método organizacional. Mas incorpore o hábito de shapear cada requisito que chega até você — refinar, questionar, rascunhar — antes de abrir a IDE. Esse hábito sozinho já transforma a qualidade das suas entregas.

Busque acesso a quem idealizou o requisito — essa é a fonte mais confiável

O primeiro passo prático para triturar um requisito é aparentemente simples, mas frequentemente ignorado: descubra quem de fato idealizou aquele pedido e converse diretamente com essa pessoa. Não com o intermediário, não com quem transcreveu a história no backlog, mas com quem teve a ideia original.

Existe um conceito poderoso por trás disso: a **fonte de maior confiabilidade**. Quando você conversa com alguém que está repassando um requisito que outra pessoa concebeu, existe um risco real de a informação estar distorcida. A pessoa intermediária pode achar que entendeu perfeitamente, pode falar com total confiança sobre o requisito, mas se não foi ela quem o idealizou, ela simplesmente não sabe o que não sabe. É como a IA generativa: ela gera respostas com total confiança, mesmo quando estão erradas, porque não tem consciência do erro.

Isso não significa que você precisa sempre participar de todo o processo de concepção. Nem sempre é possível. Mas significa que, ao receber um requisito, você deveria perguntar: “Quem pensou nisto? Posso falar com essa pessoa?” Se a resposta for “não, eu sou a pessoa que representa essa entidade”, tudo bem — mas saiba que existe um risco ali. E se existe risco, você precisa mitigá-lo com os próximos passos do processo.

“Eu aqui como uma pessoa intermediária do processo, estou te dizendo que a gente entendeu que precisa fazer isso. Mas quem foi que teve essa ideia? Foi você? Não? Então eu preciso falar com quem teve a ideia.”

Faça disso um hábito: ao receber qualquer requisito, a primeira

pergunta é “quem idealizou isso?”. Ter acesso a essa pessoa não garante que tudo vai dar certo, mas não ter acesso garante que você está operando com risco desnecessário.

Entenda o motivo de existência do requisito antes de pensar em como implementá-lo

Depois de identificar quem idealizou o requisito, o próximo passo é entender profundamente **por que** aquilo precisa ser feito. Não o “como”, ainda não — mas o “por que”. Qual problema aquele requisito resolve? Que dor do usuário ou do negócio ele endereça? Será que o que está sendo pedido é realmente a melhor solução para o problema identificado?

Esse passo é inspirado no processo de shaping do Shape Up, onde antes de definir qualquer solução, você precisa entender o problema com profundidade. Muitas vezes, um requisito chega até você já como uma solução pronta — “crie um botão aqui”, “adicione esse campo”, “faça essa integração” — sem que o problema original esteja claro. Quando você entende o motivo, você ganha a capacidade de questionar se aquela solução é de fato a melhor.

Você pode, por exemplo, descobrir que o problema pode ser resolvido de maneira mais simples, com menos código e menor esforço. Ou pode descobrir que o requisito como está descrito não resolve de fato o problema. Essas descobertas só acontecem quando você investe tempo entendendo o “por que” antes de mergulhar no “como”.

Existe também a possibilidade de você descobrir que o requisito nem deveria existir — que o problema já está resolvido de outra forma, ou que a prioridade deveria ser outra. Quanto mais cedo você faz essa descoberta, mais barato é para todo mundo.

“Eu acredito fortemente que pra você sair de uma ideia pra o código de fato que implementa aquela ideia, se você coloca muita energia

no entendimento do que precisa ser feito, você maximiza muito sua chance de codar o que de fato é necessário.”

Antes de pensar em arquitetura, design patterns ou tecnologia, pergunte: “Por que isso precisa existir? Que problema resolve?” Se você não conseguir responder com clareza, você ainda não está pronto para implementar.

Faça double check do seu entendimento — aprendizagem negativa é um risco real

Você leu o requisito, conversou com quem idealizou, entendeu o motivo. Agora precisa de um passo que parece óbvio mas quase ninguém faz: **validar se o que você entendeu é de fato o que a outra pessoa quis dizer**. Esse é o double check — ou triple check — do seu entendimento.

No mundo da aprendizagem, existe o conceito de **aprendizagem negativa**: quando você acha que aprendeu algo, mas aprendeu errado. Você também pode ter um aprendizado parcial — entendeu parte da coisa, mas a outra parte ficou nebulosa. E como você descobre isso? Através do feedback. Sem feedback, você segue em frente convicto de que entendeu, e só descobre o erro quando já é tarde demais.

Na prática, isso significa voltar para a pessoa que pediu o requisito e explicar, com suas próprias palavras, o que você entendeu. Não é reler o que está escrito no ticket — é reformular o entendimento e pedir confirmação. Você quer sair dessa conversa com a pessoa dizendo: “É exatamente isso que eu preciso.”

Se a pessoa disser “não, não era bem isso”, ótimo — você acabou de evitar horas ou dias de retrabalho. E se você não tem acesso a essa pessoa? Então você tem um problema real no seu processo de desenvolvimento. Um problema que diminui a confiabilidade de toda entrega.

“Você só quer sair dessa conversa quando você explicar para a pessoa o que ela pediu e ela falar assim: é exatamente isso que eu preciso.”

Registre o resultado do double check — mande um email, anote

no ticket, grave a conversa. Assim você tem uma referência para quando surgirem dúvidas durante a implementação e cria uma cultura de validação na sua equipe.

Rascunhe o fluxo antes de codar para revelar problemas escondidos

Depois de validar seu entendimento com quem pediu o requisito, é hora de dar mais um passo antes de abrir a IDE: **rascunhar o fluxo de implementação**. A ideia é materializar visualmente ou textualmente como o requisito se transforma em software, de uma forma que a pessoa que pediu consiga entender e validar.

Inspirado no conceito de Fat Marker Sketch do Shape Up — desenhos com traços grosseiros que mostram o contorno da solução sem entrar em detalhes visuais finais — você pode usar qualquer ferramenta: um rabisco no papel, um diagrama simples, uma descrição textual do fluxo. O importante é que a pessoa não-técnica consiga olhar e dizer “sim, é isso” ou “não, tem algo errado aqui”.

Esse exercício serve a dois propósitos. Primeiro, é mais uma rodada de reflexão — é refletindo sobre algo que você pode descobrir lacunas que não apareceram antes. “Espera, o que acontece quando o usuário não tem essa informação? E se esse serviço estiver fora do ar? Quem aprova isso?” Perguntas que só surgem quando você tenta materializar a solução.

Segundo, você cria mais um ponto de validação com quem pediu o requisito. Cada validação é uma oportunidade de fechar buracos antes da implementação. Cada buraco fechado agora é retrabalho que você evita depois.

“O que a gente quer é que na hora que você vai codar e vai entregar a parada, você esteja o mais próximo possível da versão que tem que ser colocada mesmo em produção.”

Não precisa ser bonito. Um rascunho num papel, um fluxograma

simples ou até uma descrição em texto corrido já funcionam. O objetivo não é a estética do artefato, é a reflexão que ele provoca e a validação que ele permite.

Especifique cenários de teste antes de codar — é mais uma camada de reflexão

Você já entendeu o requisito, validou seu entendimento, rascunhou o fluxo. Agora vem um passo que pode gerar resistência: **tentar especificar cenários de teste antes de escrever código**. Não é sobre adotar BDD ou qualquer metodologia específica — é sobre usar a especificação de testes como mais uma ferramenta de reflexão.

Se você consegue descrever cenários como “dado que o cliente está autenticado, quando ele solicita o saldo, então o sistema exibe o saldo atualizado”, isso é um sinal forte de que o requisito está nítido na sua cabeça. Se você não consegue especificar esses cenários, provavelmente ainda tem buracos no seu entendimento.

Essa etapa também funciona como uma rede de segurança. Ao pensar nos cenários de teste, você naturalmente considera os casos de borda: o que acontece quando o input é inválido? E quando o serviço externo está fora do ar? E quando o usuário tenta fazer algo inesperado? São perguntas que frequentemente não aparecem na descrição do requisito, mas que você vai ter que resolver na hora de codar. Melhor descobri-las agora.

A analogia com testes automatizados é direta. A indústria já aceitou que escrever testes automatizados demora mais, mas a troca é maior confiabilidade. A mesma lógica se aplica aqui: especificar cenários antes demora mais, mas a troca é um entendimento mais profundo e menos surpresas na implementação.

“Eu não estou advogando nem para BDD, nem para qualquer coisa DD. Estou apenas te dizendo que fazer isso é um ato de re-

flexão e refletir sobre isso antes de codar pode ajudar você e a sua equipe a descobrir coisas que não tinham sido observadas até este momento.”

Você pode escrever os cenários na mão ou pedir para uma IA generativa sugerir-los a partir da descrição do requisito. O importante não é quem gera — é você revisando criticamente cada cenário e usando isso como mais uma oportunidade de encontrar lacunas.

Se você não consegue imaginar o que vai codar, como vai executar?

O último passo antes de codar é talvez o mais poderoso: **descrever o passo a passo da implementação**. Não em código — em linguagem natural. A ideia é simples: se você consegue imaginar todos os detalhes do que vai fazer, provavelmente vai conseguir executar. Mas se você não consegue nem imaginar, como vai codar?

Essa técnica é inspirada no Cognitive Walkthrough, originalmente usada em testes de interface. Você faz uma caminhada mental detalhada pela implementação: “Primeiro vou criar o controller, depois defino uma classe de request com os dados, coloco as validações nos atributos, trato as regras de negócio, salvo no banco, retorno a confirmação.” Ao fazer isso, você revela partes que vão dar mais trabalho e, principalmente, partes que você não sabe como resolver.

Quando você encontra algo que não sabe resolver nesse exercício, você tem a oportunidade de buscar a resposta antes de codar — pesquisando, perguntando para colegas, consultando a pessoa que pediu o requisito. É muito melhor descobrir que você não sabe algo agora do que no meio da implementação, quando o custo de parar e repensar é muito maior.

Em tempos de IA generativa, esse passo ganha ainda mais valor: se você descreveu com precisão o que precisa ser feito, pode usar essa descrição como input para um copilot ou assistente de código e obter resultados muito mais próximos do esperado.

“Se você não for capaz de imaginar, como você vai ser capaz de executar? Não é porque você imagina que você executa, mas se você não consegue nem imaginar, como você vai fazer?”

Antes de codar cada história ou tarefa, escreva em linguagem natural o passo a passo do que você vai implementar. Se encontrar um trecho que não sabe descrever, pare e resolva essa dúvida. Esse exercício custa minutos e pode poupar horas de tentativa e erro.

Resumo Final

Engenharia de requisitos não é burocracia — é a maneira mais barata de aumentar a qualidade do que você entrega como pessoa desenvolvedora. A pesquisa científica e a experiência prática convergem no mesmo ponto: requisitos mal refinados são a principal causa de retrabalho, atrasos e falhas em projetos de software, independentemente do método utilizado.

Mensagem Central: Codar é a última etapa de um processo de reflexão. Quanto mais trabalho de qualidade você faz antes de abrir a IDE, maior a chance de acertar de primeira e menor o custo operacional da sua entrega.

O método de cinco passos para triturar requisitos: **(1)** Busque acesso a quem idealizou o requisito — a fonte mais confiável. **(2)** Entenda o motivo de existência — por que isso precisa ser feito? **(3)** Faça double check do seu entendimento — explique com suas palavras e peça confirmação. **(4)** Rascunhe o fluxo e especifique cenários de teste — materialize antes de codar. **(5)** Descreva o passo a passo da implementação — se você não consegue imaginar, não está pronto para executar.

Cada passo é uma camada de reflexão que fecha buracos antes da implementação. O custo é tempo investido antes de codar. O retorno é menos retrabalho, mais confiabilidade e entregas que de fato fazem o que precisam.

Conselho prático

Comece pequeno: escolha uma tarefa do seu backlog atual e aplique os cinco passos antes de implementá-la. Compare o resultado com o que você costuma entregar sem esse processo. Faça isso

por algumas semanas e você vai estar craque em triturar requisitos — gerando muito mais valor do que gera hoje.

Sobre este resumo

Este resumo foi gerado a partir do curso **“Engenharia de Requisitos para Devs”** de **Alberto Souza**, parte da Jornada Dev Eficiente.

O curso apresenta uma abordagem prática de engenharia de requisitos voltada para pessoas desenvolvedoras, combinando insights da engenharia de requisitos formal, do método Shape Up (37signals/Basecamp) e de pesquisas acadêmicas como o artigo “Naming the Pain in Requirements Engineering”. O resultado é um método de cinco passos para triturar requisitos antes de codar, reduzindo retrabalho e aumentando a qualidade das entregas.

Este blink foi criado para oferecer uma visão rápida e prática dos principais conceitos do curso. Para aprofundamento com exemplos detalhados e exercícios aplicados ao seu contexto de trabalho, recomendamos o curso completo na plataforma Dev Eficiente.